

Time Series

Stationnary Time Series

Mr. Aymen Ben Brik

Time Series Course 1GAMMA

7 avril 2026

Session Objectives

- Understand the concept of stationarity in time series
- Distinguish between strict and weak stationarity
- Learn how to test for stationarity (ADF, KPSS)
- Interpret ACF and PACF plots
- Introduce AR, MA, and ARMA models
- Diagnose model adequacy through residual analysis

What is a Stationary Time Series ?

- A time series is said to be **stationary** if its statistical properties do not change over time
- In particular, a stationary series has :
 - Constant mean
 - Constant variance
 - Constant autocovariance (depends only on the lag)
- Intuitively, the series fluctuates around a stable level with no systematic trend or changing variability
- Stationarity is a key assumption for many time series models (AR, MA, ARMA)

Stationary vs Non-Stationary Series

- **Stationary series :**

- Fluctuates around a constant mean
- Variance remains stable over time
- No clear trend or seasonality

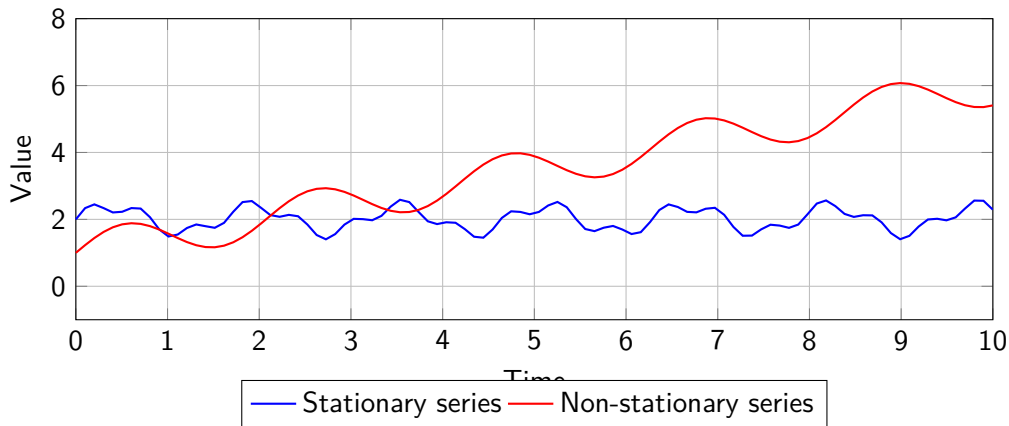
- **Non-stationary series :**

- Presence of trend (upward or downward)
- Changing variance over time
- Possible seasonality or structural changes

- **Examples :**

- Stationary : white noise
- Non-stationary : random walk, trending data

Visual Illustration : Stationary vs Non-Stationary



- The blue series fluctuates around a constant mean
- The red series shows an upward trend over time

Simulation of a White Noise Process

Definition : A white noise process is a sequence of uncorrelated random variables with zero mean and constant variance.

R

```
set.seed(123)
n <- 200
wn <- rnorm(n, 0, 1)
plot.ts(wn, col="blue",
  main="White Noise in R")
```

Python

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(123)
wn = np.random.normal(0, 1, 200)
plt.plot(wn)
plt.title("White Noise in Python")
plt.show()
```

Observation : The series fluctuates randomly around zero.

Random Walk Simulation

Model :

$$X_t = X_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim WN(0, \sigma^2)$$

R

```
set.seed(123)
n <- 200
eps <- rnorm(n, 0, 1)
rw <- cumsum(eps)

plot.ts(rw, col="red",
  main="Random Walk (R)")
```

Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)
eps = np.random.normal(0, 1, 200)
rw = np.cumsum(eps)

plt.plot(rw)
plt.title("Random Walk (Python)")
plt.show()
```

Key idea : The accumulation of shocks makes the process non-stationary.

Strong Stationarity

Definition :

A time series $\{X_t\}$ is said to be **strictly (strongly) stationary** if :

- For any integer k , any time points $t_1, \dots, t_k$, and any shift h ,
- The joint distribution of

$$(X_{t_1}, X_{t_2}, \dots, X_{t_k})$$

is identical to that of

$$(X_{t_1+h}, X_{t_2+h}, \dots, X_{t_k+h})$$

Key idea :

- The probabilistic behavior of the series does not change over time

Intuition of Strong Stationarity

Interpretation :

- The entire probabilistic structure of the series is invariant over time
- Not only mean and variance, but **all distributions** remain unchanged

Example (Stationary) :

- White noise process :

$$X_t \sim \mathcal{N}(0, \sigma^2)$$

- Same distribution at all times

Counterexample (Non-Stationary) :

- Time-varying variance :

$$X_t \sim \mathcal{N}(0, t)$$

- Distribution changes over time

Key takeaway :

Weak Stationarity (Covariance Stationarity)

Definition :

A time series $\{X_t\}$ is said to be **weakly stationary** if :

- $\mathbb{E}[X_t] = \mu$ (constant mean)
- $\text{Var}(X_t) = \sigma^2$ (constant variance)
- $\text{Cov}(X_t, X_{t+h}) = \gamma(h)$ depends only on the lag h

Interpretation :

- The first two moments are time-invariant
- The dependence structure depends only on the time gap, not on time itself

Key insight :

- Weak stationarity is sufficient for most models (AR, MA, ARMA)

Strong vs Weak Stationarity

Property	Strong Stationarity	Weak Stationarity
Distribution	Unchanged over time	Not required
Mean	Constant	Constant
Variance	Constant	Constant
Autocovariance	Implicitly constant	Depends only on lag
Conditions	Very restrictive	Less restrictive
Practical use	Rare in practice	Widely used (AR, MA, ARMA)

Key takeaway :

- Strong stationarity $\Rightarrow$ Weak stationarity (under finite variance)
- In practice, we usually assume **weak stationarity**

How to Check Stationarity (Graphical Intuition)

Visual Inspection :

- Plot the time series and observe its behavior over time

Signs of Stationarity :

- Fluctuates around a constant mean
- Constant variance (no increasing/decreasing spread)
- No visible trend or seasonality

Signs of Non-Stationarity :

- Upward or downward trend
- Changing variance over time
- Presence of seasonal patterns

Tools :

- Time series plot
- Rolling mean and rolling variance

Rolling Statistics (Mean and Variance)

Idea : Compute statistics over a moving window

R

```
library(zoo)

n <- 200
x <- cumsum(rnorm(n)) # example

roll_mean <- rollmean(x, k=20,
  fill=NA)
roll_var <- rollapply(x, 20,
  var, fill=NA)

plot(x, type="l")
lines(roll_mean, col="blue")
```

Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

x = np.cumsum(
  np.random.normal(0,1,200))

s = pd.Series(x)
roll_mean = s.rolling(20).mean()
roll_var = s.rolling(20).var()

plt.plot(s)
plt.plot(roll_mean)
plt.show()
```

Exercise :

Consider the following time series :

- ① $X_t = \varepsilon_t$, where $\varepsilon_t \sim WN(0, \sigma^2)$
- ② $X_t = t + \varepsilon_t$
- ③ $X_t = X_{t-1} + \varepsilon_t$
- ④ $X_t = \varepsilon_t + 0.5 \varepsilon_{t-1}$

Questions :

- Which of these processes are stationary ?
- Justify your answer (mean, variance, dependence structure)

ADF Test (Augmented Dickey-Fuller)

Objective :

- Test whether a time series is **stationary**

Key idea :

- Detect the presence of a **unit root**

Why important ?

- A unit root implies **non-stationarity**
- Many models (ARMA) require stationarity

ADF Test : Model

Basic idea :

Start from an AR(1) model :

$$X_t = \rho X_{t-1} + \varepsilon_t$$

Rewriting :

$$\Delta X_t = \gamma X_{t-1} + \varepsilon_t \quad \text{with } \gamma = \rho - 1$$

Augmented version :

$$\Delta X_t = \gamma X_{t-1} + \sum_{i=1}^p \alpha_i \Delta X_{t-i} + \varepsilon_t$$

Goal : Test whether $\gamma = 0$

ADF Test : Hypotheses

Null hypothesis :

- $H_0 : \gamma = 0$
- The series has a **unit root**
- $\Rightarrow$ Non-stationary

Alternative hypothesis :

- $H_1 : \gamma < 0$
- The series is **stationary**

Interpretation :

- Reject $H_0 \Rightarrow$ Stationary series
- Fail to reject $H_0 \Rightarrow$ Non-stationary

Based on p-value :

- $p\text{-value} < 0.05 \Rightarrow \text{Reject } H_0 \Rightarrow \text{Stationary}$
- $p\text{-value} > 0.05 \Rightarrow \text{Fail to reject } H_0 \Rightarrow \text{Non-stationary}$

Important :

- The test is sensitive to :
 - Lag selection
 - Trend or intercept specification

ADF Test : Implementation in R and Python

R

```
library(tseries)

x <- cumsum(rnorm(200)) # example series

adf.test(x)
```

Output :

- test statistic
- p-value

Python

```
import numpy as np
from statsmodels.tsa.stattools import adfuller

x = np.cumsum(np.random.normal(0, 1, 200))

result = adfuller(x)

print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

Output :

- ADF statistic
- p-value

ADF Test : Interpretation of Results

Decision rule based on the p-value :

- If p-value < 0.05 : reject H_0
- If p-value > 0.05 : fail to reject H_0

Interpretation :

- Rejecting H_0 means that the series is likely **stationary**
- Failing to reject H_0 suggests the presence of a **unit root**
- Therefore, the series is likely **non-stationary**

Reminder :

- H_0 : the series has a unit root
- H_1 : the series is stationary

KPSS Test (Kwiatkowski-Phillips-Schmidt-Shin)

Objective :

- Test whether a time series is **stationary**

Key difference from ADF :

- The KPSS test takes **stationarity** as the null hypothesis
- It is therefore complementary to the ADF test

Why useful ?

- Using both ADF and KPSS provides a more reliable conclusion

Null hypothesis :

- H_0 : the series is **stationary**

Alternative hypothesis :

- H_1 : the series is **non-stationary**

Interpretation :

- Reject $H_0 \Rightarrow$ evidence of non-stationarity
- Fail to reject $H_0 \Rightarrow$ the series may be stationary

Test	Null Hypothesis	Alternative
ADF	Unit root / Non-stationary	Stationary
KPSS	Stationary	Non-stationary

Practical interpretation :

- ADF rejects + KPSS does not reject $\Rightarrow$ likely stationary
- ADF does not reject + KPSS rejects $\Rightarrow$ likely non-stationary

KPSS Test : Implementation in R and Python

R

```
library(tseries)

x <- cumsum(rnorm(200)) # example series

kpss.test(x)
```

Python

```
import numpy as np
from statsmodels.tsa.stattools import kpss

x = np.cumsum(np.random.normal(0, 1, 200))

stat, p_value, _, _ = kpss(x)

print("KPSS Statistic:", stat)
print("p-value:", p_value)
```


Objective :

- Analyze the dependence structure of a time series

Key tools :

- ACF : Autocorrelation Function
- PACF : Partial Autocorrelation Function

Why important ?

- Identify the structure of time series models
- Help determine AR and MA orders

Autocorrelation Function (ACF)

Definition :

- The ACF measures the correlation between X_t and X_{t+h}

$$\rho(h) = \frac{\text{Cov}(X_t, X_{t+h})}{\text{Var}(X_t)}$$

Interpretation :

- Measures linear dependence at lag h
- Values between -1 and 1

Key idea :

- High autocorrelation $\Rightarrow$ strong dependence

What does ACF tell us ?

- How past values influence current values

Examples :

- White noise :
 - No correlation $\Rightarrow \text{ACF} \approx 0$ for all lags
- Persistent series :
 - Slowly decreasing ACF

Graph :

- ACF is typically represented as a bar plot

Partial Autocorrelation Function (PACF)

Definition :

- The PACF measures the direct correlation between X_t and X_{t+h}
- Removing the effect of intermediate lags

Key idea :

- PACF isolates the **pure effect** of lag h

Interpretation :

- Helps identify the order of AR models

Function	Measures	Use
ACF	Total correlation	Identify MA(q)
PACF	Direct correlation	Identify AR(p)

Key idea :

- ACF includes indirect effects
- PACF removes indirect effects

Simulation of an AR(2) : ACF and PACF

AR(2) model :

$$X_t = 0.6X_{t-1} - 0.3X_{t-2} + \varepsilon_t, \quad \varepsilon_t \sim WN(0,1)$$

R

```
set.seed(123)
x <- arima.sim(n = 200,
  model = list(ar = c(0.6, -0.3)))

par(mfrow = c(1,2))
acf(x, main = "ACF of AR(2)")
pacf(x, main = "PACF of AR(2)")
```

Python

```
import numpy as np
from statsmodels.tsa.arima_process import
from statsmodels.graphics.tsaplots import
import matplotlib.pyplot as plt

np.random.seed(123)
ar = np.array([1, -0.6, 0.3])
ma = np.array([1])
x = ArmaProcess(ar, ma).generate_sample(200)

fig, ax = plt.subplots(1, 2, figsize=(10,4))
plot_acf(x, ax=ax[0], lags=20)
plot_pacf(x, ax=ax[1], lags=20, method="yw")
```

Interpretation of ACF and PACF for an AR(2)

Typical pattern of an AR(2) :

- The **ACF** decreases gradually
- The **PACF** cuts off after lag 2

Why ?

- In an AR(p) process, the PACF truncates at lag p
- For an AR(2), only lags 1 and 2 have direct effects

Key takeaway :

- Slowly decaying ACF + PACF cutoff at lag 2
- This is the signature of an **AR(2)** process

Simulation of an MA(3) : ACF and PACF

MA(3) model :

$$X_t = \varepsilon_t + 0.5\varepsilon_{t-1} + 0.3\varepsilon_{t-2} + 0.2\varepsilon_{t-3}, \quad \varepsilon_t \sim WN(0, 1)$$

R

```
set.seed(123)
x <- arima.sim(n = 200,
  model = list(ma = c(0.5, 0.3, 0.2)))

par(mfrow = c(1,2))
acf(x, main = "ACF of MA(3)")
pacf(x, main = "PACF of MA(3)")
```

Python

```
import numpy as np
from statsmodels.tsa.arima_process import
from statsmodels.graphics.tsaplots import
import matplotlib.pyplot as plt

np.random.seed(123)
ar = np.array([1])
ma = np.array([1, 0.5, 0.3, 0.2])
x = ArmaProcess(ar, ma).generate_sample(200)

fig, ax = plt.subplots(1, 2, figsize=(10,4))
plot_acf(x, ax=ax[0], lags=20)
```


Interpretation of ACF and PACF for an MA(3)

Typical pattern of an MA(3) :

- The **ACF** cuts off after lag 3
- The **PACF** decreases gradually

Why ?

- In an MA(q) process, the ACF truncates at lag q
- Dependence exists only through past shocks up to lag 3

Key takeaway :

- ACF cutoff + PACF decay $\Rightarrow$ MA process
- Here : cutoff at lag 3 $\Rightarrow$ MA(3)

Summary : ACF/PACF Patterns (AR vs MA vs ARMA)

Model	ACF	PACF
AR(p)	Gradual decay	Cuts off after lag p
MA(q)	Cuts off after lag q	Gradual decay
ARMA(p,q)	Gradual decay	Gradual decay

Key identification rules :

- Sharp cutoff in PACF $\Rightarrow$ AR model
- Sharp cutoff in ACF $\Rightarrow$ MA model
- No clear cutoff $\Rightarrow$ ARMA model

Important :

- In practice, patterns are not always perfectly clear
- Use ACF/PACF together with model selection criteria (AIC, BIC)

Autoregressive Model AR(p)

Definition :

An autoregressive process of order p , denoted $AR(p)$, is defined as :

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-p} + \varepsilon_t$$

where :

- $\phi_1, \dots, \phi_p$ are parameters
- $\varepsilon_t \sim WN(0, \sigma^2)$

Key idea :

- The current value depends on its **past values**

AR(p) : Intuition

How does it work ?

- The present is a linear combination of the past

Example : AR(1)

$$X_t = \phi X_{t-1} + \varepsilon_t$$

- If $|\phi| < 1$: the process is stationary
- The series tends to return to its mean (mean reversion)

Interpretation :

- Large $\phi \Rightarrow$ strong persistence
- Small $\phi \Rightarrow$ weak dependence

AR(p) : Example

Example : AR(2)

$$X_t = 0.6X_{t-1} - 0.3X_{t-2} + \varepsilon_t$$

Interpretation :

- X_{t-1} has a positive influence
- X_{t-2} has a negative influence

Key idea :

- The structure captures dynamic dependence over time

Stationarity Condition of AR(p)

AR(p) model :

$$X_t = \phi_1 X_{t-1} + \cdots + \phi_p X_{t-p} + \varepsilon_t$$

Characteristic polynomial :

$$\Phi(z) = 1 - \phi_1 z - \phi_2 z^2 - \cdots - \phi_p z^p$$

Stationarity condition :

- The process is stationary if all roots of $\Phi(z) = 0$ lie **outside the unit circle**

Equivalent condition :

- All roots satisfy $|z_i| > 1$

Stationarity Condition : AR(1)

Model :

$$X_t = \phi X_{t-1} + \varepsilon_t$$

Characteristic equation :

$$1 - \phi z = 0 \quad \Rightarrow \quad z = \frac{1}{\phi}$$

Stationarity condition :

$$|z| > 1 \quad \Rightarrow \quad \left| \frac{1}{\phi} \right| > 1$$
$$\Rightarrow \quad |\phi| < 1$$

Interpretation :

- If $|\phi| < 1$: stationary (mean-reverting)
- If $|\phi| \geq 1$: non-stationary

Moving Average Model MA(q)

Definition :

A moving average process of order q , denoted MA(q), is defined as :

$$X_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}$$

where :

- $\varepsilon_t \sim WN(0, \sigma^2)$
- $\theta_1, \dots, \theta_q$ are constants

Key idea :

- The current value depends on the **current and past shocks**

MA(q) : Intuition

How does it work ?

- A shock affects the series immediately
- Its effect persists only for a limited number of periods

Interpretation :

- In an MA(q) process, a shock at time t influences :
 - X_t
 - X_{t+1}
 - ...
 - X_{t+q}
- After lag q , the effect disappears

Key difference from AR(p) :

- AR : depends on past values
- MA : depends on past shocks

MA(q) : Example

Example : MA(2)

$$X_t = \varepsilon_t + 0.6 \varepsilon_{t-1} + 0.3 \varepsilon_{t-2}$$

Interpretation :

- The current shock ε_t affects X_t
- The previous shock ε_{t-1} still affects X_t
- The shock ε_{t-2} also affects X_t
- Older shocks have no effect

Key idea :

- The memory of the process is finite

Stationarity of MA(q)

Important property :

- Every MA(q) process is **stationary**

Why ?

- X_t is a finite linear combination of white noise terms
- Its mean is constant
- Its variance is constant
- Its autocovariance depends only on the lag

Conclusion :

- MA models are always weakly stationary

Mean and Variance of an MA(q)

For

$$X_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}, \quad \varepsilon_t \sim WN(0, \sigma^2),$$

Mean :

$$\mathbb{E}[X_t] = 0$$

Variance :

$$\text{Var}(X_t) = \sigma^2 (1 + \theta_1^2 + \theta_2^2 + \cdots + \theta_q^2)$$

Remark :

- The variance is constant over time

Autocovariance of MA(q)

Model :

$$X_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}, \quad \varepsilon_t \sim WN(0, \sigma^2)$$

Autocovariance function $\gamma(h)$:

- For $h = 0$:

$$\gamma(0) = \sigma^2 (1 + \theta_1^2 + \cdots + \theta_q^2)$$

- For $1 \leq h \leq q$:

$$\gamma(h) = \sigma^2 (\theta_h + \theta_1 \theta_{h+1} + \cdots + \theta_{q-h} \theta_q)$$

- For $h > q$:

$$\gamma(h) = 0$$

Key property :

- The autocovariance function cuts off after lag q

Invertibility of MA(q)

Model :

$$X_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}$$

Objective :

- Express ε_t as a function of past values of X_t

Key idea :

- An MA(q) process is **invertible** if it can be written as an AR(∞) process

ARMA(p,q) Model

Definition :

An ARMA(p, q) process combines autoregressive and moving average components :

$$X_t = \phi_1 X_{t-1} + \cdots + \phi_p X_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}$$

where :

- $\varepsilon_t \sim WN(0, \sigma^2)$
- ϕ_i : AR parameters
- θ_j : MA parameters

ARMA(p,q) : Compact Representation

Using the lag operator B :

$$\Phi(B)X_t = \Theta(B)\varepsilon_t$$

where :

$$\Phi(B) = 1 - \phi_1 B - \dots - \phi_p B^p$$

$$\Theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$$

Key idea :

- ARMA combines dependence on past values and past shocks

ARMA(p,q) : Conditions

Stationarity :

- All roots of $\Phi(z) = 0$ lie outside the unit circle

Invertibility :

- All roots of $\Theta(z) = 0$ lie outside the unit circle

Conclusion :

- ARMA is well-defined when both conditions are satisfied

ARMA(p,q) : Intuition

How does it work ?

- AR part : captures persistence (dependence on past values)
- MA part : captures shocks (dependence on past errors)

Key idea :

- ARMA provides a flexible model for time series dynamics

Special cases :

- $\text{ARMA}(p, 0) = \text{AR}(p)$
- $\text{ARMA}(0, q) = \text{MA}(q)$

ARMA(p,q) : ACF and PACF

Typical behavior :

- ACF : gradual decay
- PACF : gradual decay

Key implication :

- No clear cutoff in ACF or PACF
- Harder to identify than AR or MA models

In practice :

- Use AIC / BIC for model selection

Objective :

- Check whether the fitted model is adequate

Key idea :

- The residuals should behave like **white noise**

Why ?

- If residuals contain structure, the model is not sufficient

A good model should produce residuals that :

- Have zero mean
- Have constant variance
- Are uncorrelated (no autocorrelation)

In other words :

- Residuals $\sim$ White Noise

Common plots :

- Residual time series plot
- Histogram of residuals
- Q-Q plot (normality check)
- ACF of residuals

What to check :

- No visible pattern
- No trend or seasonality
- No significant autocorrelation

Key idea :

- Residuals should have no autocorrelation

Interpretation :

- All autocorrelations should lie within confidence bounds
- Significant spikes $\Rightarrow$ model misspecification

Conclusion :

- If ACF shows structure, the model is not adequate

Objective :

- Test whether residuals are uncorrelated

Hypotheses :

- H_0 : residuals are independent (white noise)
- H_1 : residuals are autocorrelated

Decision :

- $p\text{-value} < 0.05 \Rightarrow$ reject H_0 (bad model)
- $p\text{-value} > 0.05 \Rightarrow$ good model

Residual Diagnostics in R

```
R  
model <- arima(x, order=c(1,0,1))  
  
res <- residuals(model)  
  
acf(res)  
Box.test(res, type="Ljung")
```

Goal :

- Check if residuals behave like white noise

Residual Diagnostics in Python

Python

```
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox

model = ARIMA(x, order=(1,0,1)).fit()
res = model.resid

acorr_ljungbox(res, lags=[10])
```

Goal :

- Test residual independence using Ljung-Box

Model Validation Checklist

A good model satisfies :

- Residuals $\sim$ white noise
- No autocorrelation (ACF flat)
- Ljung-Box test not rejected

If not :

- Try a different model (change p, q)
- Consider transformations or differencing

Objective :

- Predict future values of a time series

Key idea :

- Use past observations and a fitted model (AR, MA, ARMA)

Applications :

- Economics (GDP, inflation)
- Finance (stock prices)
- Energy demand

One-Step Ahead Forecast

Definition :

$$\hat{X}_{t+1} = \mathbb{E}[X_{t+1} \mid X_t, X_{t-1}, \dots]$$

Example : AR(1)

$$X_t = \phi X_{t-1} + \varepsilon_t$$

$$\Rightarrow \hat{X}_{t+1} = \phi X_t$$

Key idea :

- Future noise has zero expectation

Example : AR(1)

$$\hat{X}_{t+2} = \phi \hat{X}_{t+1} = \phi^2 X_t$$

$$\hat{X}_{t+h} = \phi^h X_t$$

Key idea :

- Forecasts converge to the mean when $|\phi| < 1$

Interpretation :

- The further ahead, the less precise the forecast

Forecast Uncertainty

Important :

- Forecasts are not exact values

Prediction intervals :

- Provide a range of possible future values

Key idea :

- Uncertainty increases with the forecast horizon

Example :

- Confidence intervals become wider over time

Forecasting in R

```
R  
library(forecast)  
  
model <- arima(x, order=c(1,0,1))  
  
fc <- forecast(model, h=10)  
  
plot(fc)
```

Output :

- Forecast values
- Confidence intervals

Python

```
from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(x, order=(1,0,1)).fit()

forecast = model.get_forecast(steps=10)

mean = forecast.predicted_mean
conf_int = forecast.conf_int()
```

Output :

- Forecast values
- Confidence intervals

Forecasting : Summary

Steps :

- Fit a model (AR, MA, ARMA)
- Check residuals
- Generate forecasts

Key idea :

- Good forecasts require a well-specified model

Important :

- Forecast uncertainty increases over time